

There are two approaches
 1. create a polynomial time verifier
 2. create a NTM that decides in poly time.
 Verifier, let TM T be the verifier for DS
 $T = "on \langle G, k, C \rangle ; \text{ where } C \text{ is a set of nodes that is the size } k \text{ dominating set.}"$

1. If C is not a set of nodes in G , reject
2. for each vertex in C , remove any adjacent nodes in G that are not in the set C .
3. If the remaining set contains a node other than in set C , reject.
4. otherwise accept.

NTM decider; let us create NTM N that decides DS.

- $N = "on \langle G, k \rangle"$
1. Nondeterministically select k nodes
 2. check the set of nodes selected dominates all vertices in G .
 3. If so, accept, otherwise reject.

Also, need to show that these run in polynomial time.

Example: CLIQUE $\in$ NP

suppose NTM A decides CLIQUE
 $A = "on \text{ input } \langle G, k \rangle, G \text{ is a undirected graph, } k \leq n = \# \text{ of vertices in the graph.}"$

1. Nondeterministically select a set C of k nodes from G .
2. Test if G contains all $\binom{k}{2}$ edges connecting nodes in C .
3. If the test passes, accept. otherwise, reject.

N runs in $O(\binom{n}{k}) = O(n^k)$

$NTIME(t(n)) = \{L \mid L \text{ is a language decided by } O(t(n)) \text{ time nondeterministic TM}\}$.

$-NP = \bigcup_k NTIME(n^k)$, k is a constant

$\therefore \text{ CLIQUE } \in \text{ NP}$

$\text{ CLIQUE } = \{ \langle G, k \rangle \mid G \text{ is an undirected graph of } n \text{ vertices which contains a } k\text{-clique or } k\text{-complete-subgraph} \}$

Definition

NP is the class of languages that has polynomial time verifiers.

Definitions

A verifier for a Language A is an algorithm V , where

$$A = \{ w \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c \}$$

Definition

Let f and g be functions, $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$, we say that $f(n) = O(g(n))$

$$\exists c > 0, n_0 \geq 0; \forall n \geq n_0: f(n) \leq c \cdot g(n)$$

when $f(n) = O(g(n))$, we say that $g(n)$ is an upper bound for $f(n)$

Little- O -notation

Let f and g be functions, $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$, say $f(n) = o(g(n))$ if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$

In other words, $f(n) = o(g(n))$ means for any constant $c > 0$ a number n_0 exists s.t $f(n) < g(n) \quad \forall n \geq n_0$

Q6. Let HALF-CLIQUE = $\{(G) \mid G \text{ is an undirected graph having a complete subgraph with at least } \frac{n}{2} \text{ nodes}\}$, where n is the number of nodes in G .

Show that HALF-CLIQUE is NP-complete.

Need to do 2 things

1. Show HALF-CLIQUE $\in$ NP
2. CLIQUE $\leq_p$ HALF-CLIQUE

Part 1. Easy

Create a verifier that verifies a certificate.

Given the certificate (the solution) verify the nodes in the clique has edges between each other.

takes $O(n^2)$

CLIQUE $\leq_p$ HALF-CLIQUE

$\langle G, k \rangle \rightarrow \langle G' \rangle$

Basically, there are three cases we need to look at.

case 1: $k \leq \frac{n}{2}$ ($n = |V|$)

In this case, directly input G to HALF-CLIQUE and return whatever the output.

case 2: $k > \frac{n}{2}$

We need to add some dummy nodes to G such that the clique in G does not change. Let's say # of dummy nodes needed to add is m .

$$m + n = 2k$$

$$m = 2k - n$$

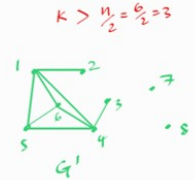
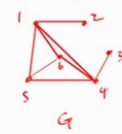
No edges needs to be added for this case.

Suppose new graph we create is G'

Then

G has a k -clique iff G' has a k -clique.

Ex: Suppose $\langle G, k \rangle \quad k = 4$



$$m = 2k - n$$

$$m = 8 - 6$$

$$m = 2$$

Case 3: $k > \frac{n}{2}$

This is slightly tricky. What we can do is to add some extra nodes, such that it will increase the size of all cliques available in G . I want to do this in a way that if there exist a k -clique then the resultant clique in G' size is greater than $\frac{n}{2}$ (n' is the # of nodes in G')

We are going to add m number of new nodes as follows

$$m + k = \frac{n + m}{2}$$

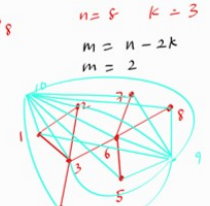
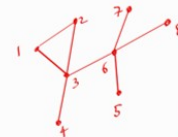
$$2m + 2k = n + m$$

$$m = n - 2k$$

for all the new nodes add edges between nodes in G and the new nodes.

Then G contains a k -clique iff G' contains a half-clique of size $\frac{n'}{2}$ ($n' = n + m$)

Ex: $G, k = 3$



G has a 3-clique if G' has $(3+2)$ -clique (5-clique)

Look at the 5-clique $(1, 2, 3, 9, 10)$ in G' it corresponds to 3-clique $(1, 2, 3)$.

3-SAT: A special case of SAT problem, where every clause has 3 literals.

Ex: $\phi = (x_1 \vee x_2 \vee \bar{x}_3) \wedge (x_4 \vee \bar{x}_5 \vee x_6) \wedge \dots \wedge (x_n \vee \bar{x}_{n+1} \vee \bar{x}_{n+2})$

$DS = \{ \langle G, k \rangle \mid G \text{ is a graph and } G \text{ has a dominating set of size at least } k \}$

Dominating Set $D \subseteq V$ s.t $G = (V, E)$ is a set D s.t

$$\forall v \in V: (v \in D) \vee (\exists u \in D \text{ s.t. } (u, v) \in E)$$

What is vertex-cover?

Given a graph $G = (V, E)$, find the minimum number of vertices that covers all edges.

Q5. Prove that $NOODD_{TM} = \{ \langle A \rangle : A \text{ is a TM and } L(A) \text{ does not contain any strings of odd length} \}$ is decidable.

We will create TM K which is a decider for $NOODD_{TM}$.

$K = \text{"on input } \langle A \rangle, \text{ where } A \text{ is a DFA:}$

1. Construct DFA B , where $L(B) = (L(A))^*$
2. Construct DFA C , where $L(C) = L(B) \cap L(A)$
3. Run the decider E_{DFA} on $\langle C \rangle$

4. If the decider E_{DFA} accepts $\langle C \rangle$, accept. If it rejects, reject.

Q7. Let $DNA_{DFA} = \{ \langle B, w \rangle : B \text{ is a DFA that does not accept } w \}$. Show that DNA_{DFA} is decidable.

Let us create TM S that decides DNA_{DFA}

$S = \text{"on } \langle B, w \rangle$

1. Simulate B on w
2. If B accepts w , reject otherwise, accept.

Q4. Prove that $NOODD_{TM} = \{ \langle M \rangle : M \text{ is a TM on } L(M) \text{ does not contain any strings of odd length} \}$ is undecidable.

create special TM M_1 , that encodes M and w . $M_1 = \text{"on input } \langle x \rangle$

1. Run M on w and accept x if M accepts w .

M accepts $w \Rightarrow L(M_1) = \Sigma^*$

M does not accept $w \Rightarrow L(M_1) = \emptyset$

We reduce $A_{TM} \leq NOODD_{TM}$. Let us assume that $NOODD_{TM}$ is decidable and TM R decides it.

Let us create TM S that decides A_{TM} . $S = \text{"on input } \langle M, w \rangle$

1. Run R on $\langle M_1 \rangle$
2. If R accepts $\langle M_1 \rangle$, reject. If R rejects $\langle M_1 \rangle$, accept.

Clearly S is a decider for A_{TM} .

$\therefore NOODD_{TM}$ is undecidable.

Q3. Prove that $2PALINDROME_{TM} = \{ \langle M \rangle : L(M) \text{ contains exactly two palindromes} \}$ is undecidable.

Note: A palindrome is a string that reads the same forwards and backwards. e.g., 11011 is a palindrome, but 1101 is not. Also, the language of M can contain any number of non-palindrome strings, but it must contain exactly two (no more, no less) palindromes.

We define the special TM M_1 that encodes M and w . $M_1 = \text{"on input } \langle x \rangle$

if $x = 00100$, accept

if $x = 11011$

run M on w , and accept x if M accepts w .

if x is anything else, reject.

M accepts $w \Rightarrow L(M_1) = \{00100, 11011\}$

M does not accept $w \Rightarrow L(M_1) = \{00100\}$

Assume TM R decides $2PALINDROME$

Let us create TM S that decides A_{TM}

$S = \text{"on input } \langle M, w \rangle$

1. Construct M_1
2. Run R on $\langle M_1 \rangle$
3. If R accepts $\langle M_1 \rangle$, accept. If R rejects $\langle M_1 \rangle$, reject.

Clearly S is a decider for A_{TM} , but this is a contradiction as A_{TM} is undecidable.

$\therefore 2PALINDROME$ is undecidable.

SET-PARTITION $\leq_P$ RECTANGLE PACKING

set-partition problem is given a set of integers $S = \{a_1, a_2, \dots, a_n\}$, find

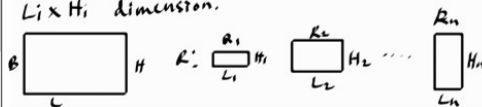
whether there exists a subset $S' \subseteq S$ s.t. $\sum_{x \in S'} x = \sum_{x \in S \setminus S'} x$

$S = \{8, 1, 11, 1, 9, 2, 3, 5\}$

$\{8, 1, 11\}$ $\{1, 9, 2, 3, 5\}$
20 20

Rectangle packing

Given a container B of size $L \times H$. You are given set of rectangles R . $R = \{R_1, R_2, \dots, R_n\}$, where R_i is of $L_i \times H_i$ dimension.



Reduction:

Let the input to the set partition be $S = \{a_1, a_2, a_3, \dots, a_n\}$

$\forall i$, construct R_i with length a_i and height ϵ

B is constructed to have length $\sum a_i$ and height 2ϵ

+ There is a solution for set partition instance iff all the R_i 's can be packed into B

$S = \{8, 1, 1, 9, 11, 2, 3, 5\}$



reduction takes $O(n)$

Q1. Suppose you have the following matrices. You want to calculate $M_1 \cdot M_2 \cdot M_3 \cdot M_4 \cdot M_5$. Use the dynamic programming algorithm you learned in the class to find the minimum number of scalar multiplications needed to calculate the final answer. Fill out the table that you are provided using the algorithm and state the minimum number of multiplications needed.

$M_1: 10 \times 25$

$M_2: 25 \times 40$

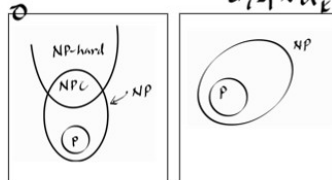
$M_3: 40 \times 10$

$M_4: 10 \times 30$

$M_5: 30 \times 15$

	1	2	3	4	5
1		0000	12500	15500	18500
2		0	10000	17500	18250
3			0	12000	10500
4				0	4500
5					0

$$m[i, j] = \min_k \{ m[i, k] + m[k+1, j] + d_{i-1} \times d_k \times d_j \}$$



$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ is a CFG and } G \text{ generates } w \}$

- Solution 02

- Dynamic programming solution

If $A \Rightarrow BC$ is a rule and $B \Rightarrow^* X$ and $C \Rightarrow^* Y$ then $A \Rightarrow^* XY$

DP solution is based on this observation.

- Define a table $[i, j]$ - which stores the collection of variables that generated the substring $w_i w_{i+1} \dots w_j$

- Idea is similar to matrix multiplication.

$w_i w_{i+1} \dots w_k \mid w_{k+1} \dots w_j$
in the table $[i, k]$ you have the variable B
in the table $[k+1, j]$ you have the variable C

Then if I have a rule $A \Rightarrow BC$ then I can add A to table $[i, j]$

Example

$w = a a b c c$

$G: S \rightarrow AB$

$A \rightarrow AB \mid a$

$B \rightarrow BC \mid b$

$C \rightarrow c$

$m[1, 2]$
 $a \mid a$
 $A A$

$+ \rightarrow A A$

	1	2	3	4	5
1		A	A		
2			A	A, S	
3				B	B
4					C
5					

$m[2, 3]$
 $a \mid b$
 $A B$

$+ \rightarrow A B$

$m[3, 4]$
 $b \mid c$
 $+ \rightarrow B C$

We will consider several languages.

$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}$

$E_{TM} = \{ \langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset \}$

$EQ_{TM} = \{ \langle M_1, M_2 \rangle \mid M_1, M_2 \text{ are TMs and } L(M_1) = L(M_2) \}$

Q4. State whether each part is TRUE or FALSE.

1. $2n + 2n^2 = O(n^{1.5})$

False

2. $2n + n \log n = o(n \log n)$

False

3. $3n + n^2 \log n = o(n^3)$

True

4. $n^2 = O(n \log^2 n)$

False